

Recovery Plan for a Legacy AmScope Microscope Camera

Bjørn Remseth
Email: la3lma@gmail.com
With AI assistance

Abstract—This document defines a staged investigation for recovering useful image capture from a circa-2010 AmScope microscope camera. Initial enumeration identifies it as a Tucsen TCA-10.0N/IS1000-family device with USB ID 0547:C003. It exposes a vendor-specific interface and a single bulk-IN endpoint rather than USB Video Class. The investigation preserved this evidence, used the recovered Windows stack as an observable reference, and reproduced its protocol in a portable libusb reader. Raspberry Pi Linux now cold-initializes the camera, streams 1280 by 960 Bayer8 frames, applies exposure and gain, and presents live video to ordinary applications through a temporary V4L2 loopback device. The same reader captures the verified 3664 by 2748 full-resolution mode. A custom camera kernel driver remains unnecessary. A renewed compatibility search finds useful sensor and V4L2 prior art, but no maintained drop-in Linux driver for 0547:C003. The first microscope-mounted capture exposed and then resolved a host-side record assembly error. The corrected reader now produces coherent 1280 by 960 and 3664 by 2748 images, monotonic optical gain response, multi-resolution stills, consecutive frames suitable for motion recording, and repeatable in-situ white balance from a neutral slide region. Final Bayer phase, known-target color calibration, a focus curve, and a post-fix Linux application retest remain open.

Index Terms—USB camera, digital microscopy, device recovery, protocol analysis, UVC, Video4Linux, macOS

version: 2026-10-04-12:03:03-CEST-abfcd0e9-main

Note on document creation: This document was created with AI assistance. See the “About This Document” and “Note on References and Verification” sections for details on the methodology and verification processes used.

Project map. This report is the narrative research record. The live Docstack tracks the evidence-gated plan and current state, while the source repository and README provide the released reader, build instructions, and application recipes. The three documents link to one another near their beginning.

I. INTRODUCTION

The subject of this project is an AmScope microscope camera purchased around 2010. It is believed to use a 10-megapixel image sensor. It shipped with a Windows driver of limited quality and has never been made reliably useful. USB enumeration now establishes that the camera does not present an ordinary USB Video Class (UVC) interface.

The practical question was whether the camera could be made useful on Linux or macOS. Matching official Windows packages and a successful reference trace have now answered the transport question: the vendor protocol can be reproduced directly through libusb, without firmware upload or a camera-specific kernel driver. The remaining uncertainty is concentrated in known-target color and focus calibration plus application packaging rather than basic device initialization, frame acquisition, or full-sensor transport.

The project will therefore proceed by evidence gates. Each gate should answer a specific question before more invasive or expensive work begins. The first goal is a repeatable still-image capture, not immediate integration with every camera application.

II. OBJECTIVES AND SUCCESS CRITERIA

The primary objective is to obtain reproducible images from the camera on at least one actively maintained operating system. Linux is the preferred first investigation platform because it exposes USB descriptors, kernel messages, and user-space USB access particularly well. macOS remains an important target and may become the preferred daily-use platform after the protocol is known.

The project has four levels of success:

- 1) **Identification:** record the exact product, USB identifiers, interface descriptors, endpoint layout, and any visible chipset markings.
- 2) **Raw acquisition:** obtain one correctly framed image outside the original application, even if it requires a command-line prototype and offline conversion.
- 3) **Usable capture:** reliably control exposure and capture full resolution images during an extended test session.
- 4) **Application integration:** expose the device through a normal Linux or macOS camera or image-acquisition interface when doing so adds practical value.

The minimum acceptable result is a documented, repeatable Linux or macOS workflow that captures full-resolution still images. Live video, automatic exposure, color correction, and virtual-camera integration are secondary goals.

III. KNOWN FACTS, UNKNOWN, AND CONSTRAINTS

The device was sold for an AmScope microscope around 2010 and was supplied with a non-standard Windows driver. The enclosure is labelled AmScope but has no visible model or serial marking in the supplied photographs (Fig. 1). The

physical camera and at least one Linux machine and one macOS machine are available. Several Raspberry Pi computers are also available and can be dedicated to USB experiments.

On macOS the camera enumerates as 10MP CMOS Camera, with USB vendor ID 0547, product ID C003, and no serial-number string. Tucsen’s official discontinued-camera table maps this exact VID/PID pair to both the TCA-10.0N and IS1000 names[1]. The matching official Windows INF files independently label the same hardware ID as “Tucsen TCA-10.0-N Camera” and “IS1000 USB Camera.” The most defensible identification is therefore an AmScope-branded member of that Tucsen hardware family; the present evidence does not determine which sales name appeared on the original package.

The most important remaining unknowns are:

- any model text retained on the original packaging, invoice, or media;
- the endpoint-zero initialization and camera-control protocol;
- whether additional firmware is uploaded by the recovered Windows software;
- confirmation by chip marking of the strongly inferred MT9J001-compatible image sensor, and identification of the USB bridge chip;
- whether the data stream contains Bayer pixels, YUV, RGB, JPEG, or a proprietary compressed format;
- whether a compatible isolated Windows environment is available for protocol capture; and
- whether an existing Linux driver or user-space project already supports the same hardware under another brand name.

There is no dedicated Windows computer available. The present workstation is an Apple M2 Max (ARM64) Mac. A normal Windows 11 ARM virtual machine is not a substitute for an x86 Windows reference host: Windows on ARM can emulate x86 and x64 user-mode applications, but kernel drivers must be built for ARM64 and are not emulated[2]. The recovered Tucsen package contains x86/x64 KMDF drivers, not an ARM64 driver.

Full-system x86-64 emulation with QEMU is now an operational reference bench. The isolated UTM Windows 10 guest runs the legacy driver and TSView7, while a second project-local native ARM64 Ubuntu guest owns the physical USB camera, records `usbmon`, and relays the device to Windows. Linux USB/IP was tested first and failed during Windows enumeration before configuration; a one-device VirtualHere userspace relay is now staged as the compatible fallback. This arrangement keeps one physical connection on the workstation while permitting Linux development and Windows reference tracing to alternate ownership. UTM’s documentation still warns that macOS cannot provide a proper hardware reset to every captured USB device[3]; physical cold-cycle claims therefore remain separate from virtual detach/attach tests.

The project-local Ubuntu VM is the preferred rapid-development USB bench. A source-built libusb program is portable to ARM and does not depend on a vendor kernel

driver. The Raspberry Pi remains the independent deployment-validation target for install, reconnect, and endurance testing. The downloaded Windows driver cannot run on either Linux host, and the architecture support of Tucsen’s current Linux SDK is unknown; neither fact blocks an independent libusb implementation.

The current AmScope MU-series support page now supplies an ARM64 Linux AmLite build dated 2026-04-13 and Windows software v4.12.31631 [4]. Both packages were downloaded into the project and extracted without execution. The Linux package uses a broad udev permission rule for vendor 0547, but neither extracted tree names C003, TCA-10.0N, IS1000, or Tucsen. More decisively, each current Windows INF enumerates 232 hardware IDs but omits `USB\VID_0547&PID_C003`. No standalone firmware image or exact-device updater was present. This is useful negative evidence, not proof that the Linux library cannot discover the device through another mechanism; compatibility remains unclaimed and no current application was run against the camera. Package hashes and the bounded static checks are retained in `evidence/firmware-discovery.md`.

The selected test bench has now been identified as a Raspberry Pi 5 Model B Rev. 1.1 named `rpios-17`. It runs 64-bit Debian 13 (trixie) with a Raspberry Pi 6.18.39+rpt-rpi-2712 kernel. Its Ethernet and Wi-Fi interfaces received 10.0.0.90 and 10.0.0.91, respectively. IPv4 packet filtering prevented initial SSH discovery, but SSH over the two link-local IPv6 addresses established access without changing the host firewall. The host presented a new SSH key relative to the historical `rpios-16.local` entry; the owner explicitly authorized a one-time connection after both hardware interfaces presented the same new key.

No firmware should be written to non-volatile storage during discovery. The camera should not be opened until external identification and software analysis have been exhausted. The recovered vendor archives should remain unchanged and cryptographically hashed before use.

IV. INITIAL DEVICE FINDINGS

The completed macOS and libusb descriptor inventory is summarized in Table I. The device is USB 2.0 high-speed and bus-powered. Its single interface is vendor-specific, with only one non-control endpoint: bulk IN endpoint `0x82`. There is no bulk or isochronous OUT endpoint, so host commands must travel through endpoint zero as control requests. The bulk-IN endpoint is the probable image-data path, although that must be confirmed by an observed capture.

Static extraction found two matching KMDF drivers and ISCapture 3.6.8. The drivers bind `0547:C003` and configure a 65,536-byte maximum transfer. ISCapture’s 32-bit `TS1000.dll` exports capture, decoding, control, and correction functions and imports the Windows file and device APIs. This points to a thin kernel transport with model-specific protocol in user space.

These findings materially improve the feasibility assessment. A libusb implementation should be able to claim the



Fig. 1. The legacy AmScope microscope camera as received. Left: the USB Type-B connector and protective black tape over the sensor/mount area. Right: the AmScope-branded side and the grey inner surface of the protective tape. No model or serial marking is visible in either view. Photographs by Bjørn Remseth, 2026-09-26.

TABLE I
Observed USB identity and interface

Field	Observed value
Product string	10MP CMOS Camera
Manufacturer string	123456789
VID:PID	0547:C003
Serial string	absent
USB link	USB 2.0, 480 Mbit/s
Interface	0, class 0xff
Endpoint	0x82, bulk IN
Max. packet	512 bytes

vendor-specific interface on Linux or macOS and receive data from 0x82; the hard part is reconstructing the control sequence and image format. Tucsens current download page lists a Linux SDK[5], but compatibility with this discontinued device has not been established and must not be assumed. Detailed observations, source URLs, hashes, and the distinction between evidence and inference are retained in `evidence/device-identity.md`.

Native Linux enumeration on `rpilos-17` reproduced the identity and topology at USB path 1-2. The camera negotiated 480 Mbit/s, had no bound driver, created no V4L2 node, and logged no reset or driver error. The existing video nodes belonged to the Pi's platform hardware, not 0547:C003. Full output is retained in `evidence/raspberry-pi-enumeration.md`.

A historical application-level precedent now strengthens the integration case. A 2014 University College London thesis names the exact Tucsens TCA-10.0-N, Micro-Manager 1.4 live view, and an 80-hour time-lapse acquisition workflow[6]. The public Micro-Manager adapter tree near that date contains generic TWAIN and OpenCV camera adapters but no legacy Tucsens-specific adapter. Consequently the thesis proves practical use from a scientific applica-

tion, but not whether the route was TWAIN, a system-camera source, or unpublished integration. The bounded source and commit analysis is preserved in `evidence/micromanager-application-precedent.md`.

A purpose-built AArch64 libusb fixture opened the camera and claimed and released interface zero. Its audited imports contain no transfer or reset function, and its udev rule targets only 0547:C003; no vendor request was sent. See `evidence/safe-libusb-probe.md`.

Cross-architecture driver analysis confirms that `IOCTL 0x222059` hard-codes `bmRequestType=0xc0` (vendor/device/IN), then copies the DLL record's request, value, and index into the USB setup packet. The DLL sends request b3 before aligned bulk reads and searches for ten 0x88 or 0x99 bytes. Packet construction is confirmed; the request and marker meanings remain inferences documented in the evidence notes.

Subsequent analysis selected the older TCA/TSView7 software lineage as the closer match for this unit. A byte-accurate offline x86 emulator now recovers all five fixed-mode initialization plans, corrects the register-read setup packet so that the address occupies `wIndex`, and identifies a GRBG Bayer-to-BGR24 vendor memory convention. Portable C17 modules describe these plans, execute them through a fail-fast transport abstraction, assemble exact frames, and convert recorded Bayer data without importing a USB API. Golden tests pass on macOS and natively on the Raspberry Pi.

A transport-neutral reader session now builds on those modules. It enforces the planned lifecycle, exact raw length and decoded metadata, explicit latest-frame or lossless bounded queues, validated exposure/gain/frame-speed controls, structured faults, and close/reopen recovery. Its replay-only camera-reader CLI provides mode inspection and sends recorded frames through the complete session and queue contract without linking libusb. Strict native tests pass

on macOS and Ubuntu ARM64, sanitizer execution passes, and the live-captured mode-4 fixture produces byte-identical RGB24 output on both. This is a verified offline D70 slice. A deliberately narrow libusb backend now connects that session to the already-demonstrated warm-state mode-4 data plane. It verifies the exact device, vendor interface, bulk-IN endpoint, and packet size; refuses to detach a kernel driver; performs only the fixed four-read frame schedule; preserves raw bytes; and then uses the common decoder and queue. Native AArch64 Linux and Apple Silicon builds pass an exact imported- libusb allowlist and inert-token tests. Fault-injection tests additionally exercise the exact four-read schedule, short, zero-length, timeout, raw-sink, partial-evidence, no-retry, and state-guard paths on both architectures. At that stage it was not executed against the cold camera because its explicit precondition was an already streaming Windows handoff. Full source, build hashes, and the execution boundary are retained in `evidence/warm-reader-backend.md`.

A separately compiled operable extension now fills the corresponding live- control implementation seam without changing the read-only binary. It shares the warm reader's claimed handle and executes only the checked exposure, normalized-gain, or eleven-write/two-delay frame-speed plans through the fail-fast executor. Injected tests on Apple Silicon and AArch64 Linux verify the exact tuples and delays, configuration guards, transport timeouts, short responses, bad acknowledgements, delay failures, detailed step reports, and no speed-state update after a partial plan. The extension imports exactly one USB capability beyond the warm reader, `libusb_control_transfer`, and is exposed only through a separately named, inert-by-default command that accepts one prevalidated exposure, gain, or speed plan and never starts acquisition. The command and backend build and pass static/inert tests on both architectures. These earlier components supplied implementation evidence; later trace-derived physical control proof is summarized below. See `evidence/operable-control-backend.md`.

A targeted headless Ghidra 12.1.4 pass now independently corroborates this model. It resolves the matching driver's opaque KMDF function-table slots, confirms ordinary USB-target creation, descriptor retrieval, single-interface configuration, pipe enumeration, synchronous endpoint-zero control transfers, and split bulk URBs of at most 0x80000 bytes. It also confirms that the DLL's ignored IOCTL 0x22208a has no handler in the matched driver and therefore cannot conceal a missing USB transaction. No firmware-loader call appears in the driver's prepare-hardware path. Reproducible scripts, project hashes, and bounded decompiler outputs are retained in `evidence/ghidra-targeted-decompilation.md`. These static results validated the portable code independently before the cold-start trace was captured. The portable startup core now also emits all 80 even-register signature reads and validates their recovered expected values with first-mismatch diagnostics; its tests pass on macOS and the Ubuntu ARM64 bench without importing libusb.

The control surface is also separated into hardware and host processing. Exposure is expressed in milliseconds and becomes a write to sensor register 0x3012; normalized analog gain 0...320 becomes a write to 0x305e; and four internal frame-speed values select a guarded PLL sequence. Automatic exposure and white balance, RGB processing gains, gamma, contrast, saturation, color enhancement, mirrors, monochrome, and illumination frequency remain host-side algorithm or image-processing state in isolated setter tests. Earlier bounded Linux trials reached a reproducible timeout, and the first standalone register-read trial had its value and index reversed. Those conservative boundaries have now been crossed using the complete Windows-reference trace: Linux cold-initializes the physical camera, applies trace-confirmed exposure and gain writes, and captures continuous 1280 by 960 Bayer8 frames. A temporary V4L2 loopback presents converted YUYV frames to ordinary consumers; FFmpeg successfully consumed a bounded three-frame run. The taped sensor means this validates transport and framing rather than optical quality or Bayer-phase calibration.

V. EXISTING LINUX DRIVER CANDIDATES AFTER SENSOR IDENTIFICATION

The completed trace makes a much sharper compatibility search possible. The camera is not merely a device with Tucsens vendor ID: it is a vendor-specific endpoint-zero/bulk-IN design whose successful session uses the b3/b4/b5/b7/bb request families. Its mode table contains 3664 by 2748 Bayer8, while sensor-facing writes use coarse-exposure register 0x3012 and gain register 0x305e. Together, the exact full-mode geometry and Aptina/Micron register vocabulary make an MT9J001-compatible 10-megapixel imager the best-supported sensor identification. This remains an engineering inference because the camera has not been opened to read the chip marking.

The candidate comparison separates USB-driver compatibility from reusable sensor and application-layer knowledge. Sharing vendor 0547, a sensor family, or an application API is not sufficient reason to bind a driver or replay its control requests:

Linux uvcvideo — not compatible.

C003 exposes class ff, not UVC VideoControl/VideoStreaming descriptors. V4L2 is useful only above the userspace reader.

In-tree gspca_touptek — not directly compatible.

It binds 0547:6801 and uses requests 0x16/0x0b plus 16-KiB chunks, unlike C003[7]. It remains the closest kernel architecture example and a source for related exposure, Bayer-gain, PLL, crop, and size registers.

Official ToupCam SDK/Python — related family, C003 not claimed.

The lead that many AmScope cameras are rebadged ToupTek units is valid for part of the product line, but the enclosure brand and 10-megapixel resolution do not make this particular unit an MU1000. Static inspection of ToupTek's official stable

2026 and legacy 2025 SDKs found 1,768 and 1,600 explicit Windows USB pairs respectively. Both list related 0547 products, including 6801, 6010, and C010; neither lists 0547:C003[8]. The Linux udev rule grants vendor-wide permissions and is not a compatibility table. The shipped Python module is a thin `ctypes` wrapper around the proprietary native library, not an independent USB driver. The SDK API is useful inspiration for a stable frame/control interface, but it supplies no reusable C003 transport. A future older archive becomes a test candidate only if its device table explicitly names C003.

Current TUCam Linux SDK — C003 not claimed.

A published current `tuusb.conf` lists modern IDs plus 0547:A008/E004, but not C003[9]. It is relevant only if Tucsen supplies documented legacy support.

Micro-Manager TUCam adapter — not a transport driver.

The current adapter is Windows x64 and links proprietary `TUCam.lib`[10]. It supplies application- integration ideas, not an independent Linux USB path.

Historical uvscopetek — exact inventory, different driver.

Its inventory names IS1000 C003, but its implemented transport targets 0547:4d88[11]. It corroborates identity and the broader control-plus-bulk architecture.

ArduCAM MT9J001 files — sensor reference only.

The exact 3664-by-2748 mode and matching sensor registers target ArduCAM controller boards, not the Tucsen bridge[12]. They are the best open source found for reviewing full-resolution timing, crop, PLL, and exposure plans before translating individual writes.

This project's libusb reader — direct use verified.

The trace-derived C003 implementation has passed physical Pi cold start, live stream, exposure/gain, and V4L2 consumer tests. It is the current Linux path and the portable basis for macOS.

The search therefore changes what can be reused, not the chosen architecture. There is no maintained drop-in Linux driver for 0547:C003 in the sources examined, and adding the ID to the Touptek or UVC tables would not implement the missing bridge protocol. The safe path is to stabilize the existing user-space reader, keep the generic V4L2 loopback as the application boundary, and review ArduCAM's MT9J001 register plans one operation at a time when adding the full-resolution mode. A native GSPCA driver is a later packaging option only if user-space latency or deployment proves inadequate. Search commits, exact exclusions, and the distinction between direct support and reusable prior art are preserved in `evidence/existing-linux-driver-candidates.md`.

VI. INVESTIGATION PLAN

A. Phase 0: Preserve the Evidence

Photograph every label and connector, and record the apparent model number, serial number, optical mount, cable type, and any accessories. The original CD, installer, and

manual are unavailable after more than a decade and are not assumed recoverable. Matching legacy packages downloaded from Tucsen are the reference copies for this investigation. Their source URLs and SHA-256 hashes must be retained with the project.

The recovered Windows software should be treated as untrusted legacy code. Run it on a disposable or isolated Windows installation with no valuable data. Keep the downloaded archives unchanged and make all experiments from extracted working copies.

B. Phase 1: Identify the USB Device

Connect the camera directly to a Raspberry Pi USB port, avoiding an unpowered hub for the initial inventory. Capture kernel messages and descriptors before installing or binding experimental software. The following commands are read-only; the device path and identifiers discovered by the first commands will replace the placeholders in later commands.

Listing 1. Initial Linux inventory (read-only)

```
uname -a
lsusb
lsusb -t
sudo dmesg --follow
sudo lsusb -v -d VENDOR_ID:PRODUCT_ID
usb-devices
v4l2-ctl --list-devices
```

On macOS, collect the corresponding inventory without installing a driver:

Listing 2. Initial macOS inventory (read-only)

```
sw_vers
system_profiler SPUSBDataType
ioreg -p IOUSB -l -w 0
```

Record the state both before and after connection. If the device disconnects and reappears with a different product ID, that is strong evidence of a host-loaded firmware or mode-switch sequence. If its interface class is 0x0e, test UVC support immediately. A vendor-specific class (0xff) does not prove that the device is unsupported; many older camera bridges use vendor-specific control transfers with bulk or isochronous image endpoints.

Gate 1 deliverable: a device identity sheet containing photographs, all USB identifiers and strings, complete descriptors, interface and endpoint types, USB speed, and connection logs from both operating systems.

Current status: Gate 1 is complete. macOS and Raspberry Pi Linux enumeration agree on the device identity, speed, vendor-specific interface, and bulk-IN endpoint. Photographs, hashes, vendor-package identification, Linux kernel messages, sysfs properties, udev properties, and V4L2 mapping have been preserved. Neither operating system binds a camera driver.

C. Phase 2: Search for Existing Support

Search using the exact vendor/product pair, model strings, driver file names, and any chipset names. Check the Linux USB ID database, current kernel source, Video4Linux and

GSPCA drivers, libcamera support, libusb-based camera projects, archived AmScope software, and support for visually identical cameras sold under other brands. Binary driver strings and Windows INF files often reveal alternate product IDs and chipset families.

Test existing support before writing code. On Linux, note which kernel module claims each interface and whether a `/dev/video*` node appears. Query formats and controls with `v4l2-ctl`; try a single-frame capture before trying continuous video. On macOS, check whether the camera appears in a standard capture application and in the operating system's media-device inventory.

Gate 2 decision: if an existing maintained driver works, document and package the configuration. If support is close but incomplete, prepare a small upstream patch. Continue to protocol analysis only if no adequate implementation exists.

D. Phase 3: Establish the Windows Reference Behaviour

If static analysis does not reveal enough of the protocol, install the preserved vendor package in a disposable x86-64 Windows guest under full-system QEMU emulation. Do not use a Windows ARM guest, because it cannot load the legacy x86/x64 kernel driver. Configure an emulated USB 2.0 controller, attach the camera only after the guest has booted, and record the device manager identity, selected INF section, driver service, installed files, registry settings, application version, and every supported resolution and control. Failure under emulation is not evidence that the physical camera is defective; USB reset or pass-through semantics may be responsible.

Capture USB traffic while performing one action at a time:

- 1) connect the idle camera and start the vendor application;
- 2) request the lowest resolution and capture one still image;
- 3) start and stop preview at one resolution;
- 4) change only exposure, gain, white balance, and resolution in separate runs; and
- 5) request the advertised maximum-resolution still image.

USBPcap with Wireshark can provide a chronological record of control, bulk, and isochronous transfers. Process and file monitoring can reveal a separate firmware image or calibration file loaded by the application. Correlate each user action with its USB transfers; avoid a single long trace containing many unlabelled changes.

Gate 3 deliverable: a minimal annotated trace that covers device initialization, one control change, and one complete image or frame.

E. Phase 4: Build a Protocol Model

Separate the observed behaviour into a control plane and a data plane. For the control plane, document request type, request number, value, index, payload, ordering, timing, and response. For the data plane, determine the transfer type, frame boundaries, headers, dimensions, stride, bit depth, pixel order, compression, and any per-frame metadata.

Use small differential experiments. Two traces that differ only in exposure are much easier to interpret than traces

of unrelated sessions. Compare frame sizes with candidate uncompressed formats and inspect the byte distribution. A Bayer mosaic may be recognizable after reshaping at the expected dimensions; JPEG data normally has recognizable markers; a fixed header or packet sequence may expose frame boundaries.

If the host uploads firmware, extract and hash the exact payload, determine whether it is merely volatile RAM firmware, and reproduce the upload only in a user-space experiment. Do not attempt to alter persistent device firmware.

Gate 4 deliverable: a short protocol specification and a replayable capture fixture with known inputs and expected output hashes.

F. Phase 5: Implement the Smallest Useful Solution

Choose the first viable option in this order:

- 1) **Configuration only:** use an existing standard or open-source driver, possibly after a known firmware-loading or mode-switch step.
- 2) **Linux user-space prototype:** build natively in the project-local ARM64 Ubuntu VM, then reproduce on a Raspberry Pi, and use libusb to initialize the device, set conservative controls, capture one frame, and save raw bytes plus metadata. Add decoding and a standard image output only after raw capture is stable.
- 3) **Linux integration:** wrap the proven user-space acquisition in a command-line tool or bridge it to V4L2/libcamera. A kernel driver is justified only when isochronous performance, latency, or application compatibility cannot be achieved safely in user space.
- 4) **macOS implementation:** port the proven USB and decoding layer with libusb or the supported macOS USB APIs. First provide still-image capture; add a system camera extension or application plug-in only if normal camera applications must consume live video.

Keep transport, protocol, decoding, color processing, and operating-system integration as separate modules. This allows recorded USB payloads to test the decoder without requiring the physical camera.

G. Phase 6: Validate and Document

Validate device reconnection, repeated still capture, all safe resolutions, exposure and gain controls, color-channel order, dropped or corrupt frames, memory usage, and at least a one-hour stability run. Compare output from the new implementation with the Windows reference using the same microscope target and illumination. Record known limitations such as unsupported controls, reduced frame rate, required firmware, or a dependency on a particular USB controller.

The final package should include installation instructions, a permissions rule for Linux if needed, diagnostic commands, sample output, protocol notes, licenses for redistributed components, and a recovery procedure that returns the host to its previous state.

Listing 3. Proposed project artifacts

```
microscope-window-sensor/
  evidence/
    device-identity.md
    hashes.sha256
    usb-descriptors/
    logs/
  traces/
    README.md
  protocol/
    protocol.md
  tools/
    capture/
    decode/
  samples/
    README.md
  tests/
```

VII. RECOMMENDED REPOSITORY LAYOUT

The reusable implementation has now been separated from this evidence-heavy research project and published as the public `tucsen-tca-camera` repository:

<https://github.com/la3lma/tucsen-tca-camera>

Its early `v0.1.0-alpha.1` release contains the Linux userspace reader, protocol notes, bounded exposure and gain controls, optional V4L2 wrapper, udev rule, and static/inert tests. Vendor binaries, proprietary firmware, raw USB traces, and bulky laboratory evidence remain outside that repository. This report project therefore remains the reproducible lab notebook, while the public repository is the small distributable implementation intended for other owners of compatible hardware.

Evidence and code should be kept separate so that large binary traces do not accidentally enter source control. A proposed layout is:

The `traces` and `samples` directories should contain metadata and acquisition instructions in Git. Downloads, extracted vendor binaries, large captures, and other working material remain inside this project under the `gitignored work/` directory unless licensing and privacy have been reviewed. Every trace should have a companion note with host, operating-system version, USB topology, action performed, timestamp, and SHA-256 hash.

VIII. RISK ASSESSMENT

The main technical risks are an unavailable Windows reference environment, host-loaded firmware hidden inside an old executable, undocumented proprietary compression, unstable isochronous transfers, or a sensor that requires calibration data. These risks affect effort but do not automatically require a kernel driver.

There is also a risk of spending substantial time reimplementing hardware that can be replaced cheaply. After Gate 2 and again after Gate 4, compare the remaining engineering effort with the cost and quality of a supported modern camera. Continuing can still be justified for learning, preservation, or

special optical compatibility, but that should be an explicit decision.

IX. CURRENT LINUX MILESTONE AND NEXT WORK

The Raspberry Pi-hosted VirtualHere/Windows reference run has now supplied the missing cold-start evidence. The sealed `usbmon3` capture contains 744 paired operations: 25 standard controls, 191 vendor controls, and 528 bulk-IN transfers, with no pairing gaps or annotator validation issues. The apparently failing endpoint-zero results are trace-confirmed pipe side effects of the legacy driver’s hard-coded IN-direction setup packets; reproducing the same sequence brings the physical device to its streaming state.

A narrow Linux `libusb` reader now sends the seven trace-confirmed mode-2 startup controls and requests complete device records. It validates the leading ten-byte `0x88` signature, performs mode-specific record assembly, and produces 1280 by 960 Bayer8 frames. It also maps bounded exposure and gain values to register `0x3012` and `0x305e` writes. Physical Raspberry Pi tests captured multiple frames after setting 400-ms exposure and normalized gain 20. The original device bytes and application Bayer bytes are preserved separately.

The live V4L2 bridge loads a generic `v4l2loopback` device only when needed, streams converted 1280 by 960 YUYV frames into it, and removes the temporary device on exit. `v4l2-ctl` reported the intended geometry and format, and FFmpeg consumed three frames into a lossless test recording. This established the Linux application boundary without a camera-specific kernel module; a live retest with the corrected complete-record reader remains pending when the camera returns to Linux.

The distributable implementation is published at `la3lma/tucsen-tca-camera`. The exact public commit used on the Pi passed a 30-frame controlled capture and ten independent open, initialize, ten-frame capture, and close processes—100 of 100 frames—without a transport failure. A 14,400-frame run at 100-ms exposure then completed in 2,410 seconds with exit status zero and `frames=14400 status=ok`; sampled resident memory remained 4,784 KiB. GitHub CI builds and audits the reader on both Ubuntu and macOS. A subsequent hardening change was physically observed to flush the complete 1,229,312-byte untouched first frame while a 100-frame reader process was still active.

Full-resolution work has also crossed its physical gate. The recovered mode table maps selector `b4/c0` to 3664 by 2748 Bayer8. The capture routine calculates its device length as $((\text{pixels} \gg 9) + 1) \ll 9$: 10,068,992 bytes. Optical evidence later corrected the original read-boundary interpretation: the host must issue 10,068,992-byte record requests. A mode-0 frame consists of 10,068,480 bytes at offsets 512–10,068,991 of record (N) plus a 192-byte continuation at offsets 320–511 of record (N+1). Mode 2 fits wholly after a 512-byte prefix in one 1,229,312-byte record. Separate 524,288-byte requests each restart at a new record origin; concatenating them caused repeated scene regions and a false seam.

Public main commit `a61c044` packages the remaining optical measurements as a reproducible procedure and adds `tca-frame-stats`, a Python-standard-library utility for one headerless Bayer8 frame. It records exact geometry and SHA-256, intensity percentiles and clipping, the four sensor-parity planes, and a phase-independent Laplacian focus score over 2-by-2 Bayer-tile means. Ubuntu and macOS CI passed. The exact commit also rebuilt on the Pi, passed every test, and cold-captured a fresh 1,229,312-byte device frame whose 1,228,800-byte Bayer payload produced a valid JSON report. The sensor was still covered, so this proves the release tooling and data path, not Bayer phase, color, orientation, or focus. Commands, hashes, and the acceptance distinction are retained in `evidence/optical-validation-tooling.md`.

Follow-up public commit `fb502c1` closes the remaining software-only package-operations gap. It documents installation under `PREFIX` and optional `DESTDIR`, removes exactly `tca-camera`, `tca-v4l2`, and `tca-frame-stats`, and leaves optional `udev` and distribution-managed loopback components untouched. A staged test proves that an unrelated same-directory sentinel survives removal. The full suite passes on Apple Silicon, the Pi, and Ubuntu/macOS CI; see `evidence/package-operations.md`.

The combined changes are published as annotated pre-release `v0.2.0-alpha.3` at exact commit `3c16d2a`. Independent main and tag CI runs pass on Ubuntu and macOS. A clean exact-tag Pi worktree passed the complete suite, then cold-opened the physical camera, applied gain and exposure, captured one exact-size preview frame, analyzed it, and exited zero. The release notes and `evidence/public-release-alpha3.md` explicitly retain the cable-reconnect and microscope-optics gates.

Native-cable Apple Silicon validation now closes a separate portability deferral. With the camera connected directly to the workstation, IOKit reported one active 10MP CMOS Camera at `0547:c003` and 480 Mbit/s. The exact public alpha-3 tag rebuilt and passed its test suite, then captured three 1280 by 960 preview frames, one 3664 by 2748 full-resolution frame, and a fresh-process preview frame. Every successful run applied gain and exposure, produced the then-expected byte schedule, and returned exact byte counts, and exited zero without Windows or the Pi relay. Covered-sensor statistics had medians 10–11 on an 8-bit scale, as expected for transport validation rather than an optical test.

The first direct invocation also supplied a useful fault observation. It failed closed when `b5/a2` returned a one-byte data stage instead of the reference stall; a bounded diagnostic pass and all later reader invocations returned the expected stall for every startup command. The event is retained as a self-clearing cold-attach transient. It does not justify accepting arbitrary short control responses or adding a broad retry. Exact commands, logs, byte counts, statistics, and hashes are recorded in `evidence/macos-native-cable-validation.md`.

The next cable reconnect placed the camera on its intended microscope optical path. The first alpha-3 rendering showed dust and debris but also repeated scene regions, a



Fig. 2. Corrected 1280-by-960 microscope frame at 100-ms exposure and gain 256, rendered with the provisional GRBG phase. Dust, fibers, and circular features are spatially coherent across the full frame; the former horizontal split and repeated quadrants are absent.

hard horizontal boundary, and a dark lower region. Its Bayer payload had autocorrelation 0.983 at an exact 524,288-byte lag. The apparent four-region sensor layout was therefore a host assembly artifact: every separate 524,288-byte bulk request restarted at the beginning of a new camera record.

A one-request diagnostic probe first resolved the preview format. A mode-2 record is 1,229,312 bytes: a 512-byte prefix beginning with ten `0x88` bytes, followed by the complete 1,228,800-byte Bayer raster. A longer mode-0 request then exposed the next `0x88` marker at exactly byte 10,068,992. The apparent 320-byte prefix actually contains 320 metadata bytes followed by the 192-pixel continuation of the preceding frame; the current frame begins at offset 512 and is completed by that continuation in the next record. A one-record look-ahead now produces every one of the 10,068,672 pixels without the false left strip. The seamless preview appears in Fig. 2, and the assembled full-resolution image appears in Fig. 3.

The blank illuminated field also provides a useful in-situ neutral reference. Robust medians from clean Bayer regions independently yielded red, green, and blue multipliers (0.8788, 1.0, 1.9773) in mode 0 and (0.8624, 1.0, 2.0) in mode 2. Figure 4 shows the latter correction. This incorporates the tungsten lamp's actual temperature, condenser, optics, filter mosaic, and sensor response. It makes the reference field chromatically neutral but does not raise exposure or prove the final Bayer phase; a known color target remains necessary for colorimetry.

The optical session also validates brightness control. The camera retains one pre-control frame in its buffer; the reader now consumes that record before publishing frame zero. At fixed 100-ms exposure, mean raw intensity increased monotonically from 14.67 at gain 0 to 154.88 at gain 320. Gain 256 yielded mean 83.23, median 91, p99 125, and only six saturated pixels per million. A subsequent 100-ms/gain-256



Fig. 3. Downscaled view of a complete 3664-by-2748 mode-0 frame assembled across records (N) and (N+1). The former 192-pixel left strip is absent; all pixels are retained rather than cropped.



Fig. 4. The 1280-by-960 field after neutral-background white balance. The measured red/green/blue multipliers are approximately (0.86/1.0/2.0), removing the tungsten/sensor yellow-green cast while preserving brightness.

run captured 16 distinct coherent frames in 2.03 seconds including initialization and the settling record. FFmpeg rendered the same data as 1280-by-960, 800-by-600, and 640-by-480 stills and encoded a 640-by-480 H.264 proof clip at a selected 6-fps playback time base. Raw Bayer contains no timestamps, so that playback rate is not yet a formal hardware frame-rate measurement. Evidence and hashes are retained in `evidence/first-optical-capture.md`.

Changing the microscope objective to 10x materially changed the light level. Reusing the previous 100-ms/gain-256 settings clipped 99.6 percent of preview pixels and 98.5 percent of full-resolution pixels, so those frames cannot be used to judge focus. A bounded gain-zero sweep from 1 to 250 ms found a clean working point at 250 ms. The preview then had mean 153.60, median 168, p99 211, and



Fig. 5. Unclipped 1280-by-960 preview at 10x, 250-ms exposure, and gain 0, after neutral-background white balance. This is direct evidence that the reader responds correctly to a changed optical configuration; the final Bayer phase remains provisional.

no saturated pixels; the complete full-resolution still had mean 141.57, median 153, p99 206, and no saturated pixels. Figure 5 records the resulting 10x field. Individual fibers and particles are visibly resolved, but no numerical sharpness comparison with the earlier objective is claimed because magnification, scene content, exposure, and possibly the binocular/camera focal planes all changed together. A subsequent 12-frame run produced 12 distinct preview hashes in 2.43 seconds including initialization and the settling record, and encoded cleanly as a 640-by-480 H.264 proof clip. Its selected 4-fps playback time base is not a measured camera timestamp.

A. Field Illumination Uniformity

Figure 5 is brighter in the center and upper field than at its lower and left edges. This is not well described by a purely radial sensor falloff: equal 256-by-192 analysis regions have mean luma 178.3 at the center, 193.9 at top-middle, 149.8 at bottom-middle, and 134.0 at bottom-left. The top-middle/bottom-left ratio is therefore about 1.45. The asymmetry points first to lamp, condenser, or relay alignment, although normal cosine falloff and an undersized camera image circle can contribute. These measurements characterize this one frame, not a calibrated uniform target.

The preferred remedy is optical. Köhler illumination deliberately images the lamp filament into the condenser aperture rather than the specimen plane; proper filament and condenser centering should produce a bright, even field even from a tungsten filament[13], [14]. The following checklist should be applied with a clean blank slide and the camera preview visible:

- 1) Focus the specimen, close the field diaphragm, raise or lower the condenser until the diaphragm edge is sharp, center it with the condenser screws, then open it only just beyond the camera field.

- 2) View the objective rear aperture if possible. Center and focus the lamp filament/collector image so it fills the condenser aperture evenly, then set the condenser aperture to roughly 70–80 percent of the objective rear aperture[14]. Do not use that iris as the primary brightness control.
- 3) Check the microscope’s documented low-power condenser arrangement for the 10x objective. Some condensers require lowering or swinging out a top lens to illuminate a wide low-power field.
- 4) Reseat and center the photo port, C-mount adapter, and relay lens. Confirm that the relay is intended for the sensor size and that no filter, reducer, extension, or partly inserted stop clips the image circle.
- 5) Clean accessible condenser, field-lens, objective, and adapter surfaces. After uniformity is aligned, use lamp voltage or a heat-safe neutral-density filter for brightness rather than misadjusting the field or aperture diaphragms.
- 6) Rotate only the camera/adapter by 180 degrees and capture another blank. A pattern that rotates implicates the camera relay/adapter; a pattern fixed in microscope coordinates implicates illumination. Repeat at another objective to separate a 10x coverage problem from a general alignment problem.

Residual shading is best handled with calibrated flat-field correction rather than local contrast enhancement. For each Bayer color plane c , acquire an average dark frame D_c with light blocked and an average or median flat F_c from a blank, defocused or translated field at the same objective, condenser, lamp voltage, exposure, gain, full sensor area, and binning. Then apply, before demosaicing,

$$C_c(x, y) = (I_c(x, y) - D_c(x, y)) \frac{\text{median}(F_c - D_c)}{\max(F_c(x, y) - D_c(x, y), \epsilon)} \quad (1)$$

Separate R, G1, G2, and B maps avoid mixing spatial shading with white balance; global white balance follows the flat correction. Averaging 16–64 darks and flats suppresses temporal noise. Micro-Manager documents the same essential background-subtraction and flat-division model and recommends many full-chip, unbinned calibration images[15]. A smooth two-dimensional surface fitted to the flat corrects illumination while retaining fixed dust; the unsmoothed flat can also suppress stationary dust shadows. Because this field is asymmetric, a two-dimensional map is preferable to a radial-only correction. Large-kernel background division can approximate a flat when none exists, while rolling-ball subtraction, homomorphic filtering, or CLAHE may make an image look more even but can alter quantitative intensity. Originals, calibration files, settings, and hashes should therefore be retained, and recalibration should follow any optical-path change. The same precomputed map can run in user space before RGB conversion or V4L2 output; no kernel driver change is required.

That path is now implemented in public commit `1dc096b`. The portable C17 `tca-flat-field` tool records explicit

geometry and Bayer phase with a fixed-point dark level and gain for every pixel, normalizes R, G1, G2, and B independently, and filters complete Bayer frames before demosaic. An optional `TCA_FLAT_FIELD` setting inserts the filter into the Linux V4L2 bridge. A deterministic Raspberry Pi preflight corrected six 1280-by-960 GRBG frames, converted them to YUYV, and had a separate `v4l2-ctl` process consume all six through a temporary loopback device; cleanup removed the device and module. Sixty preview frames were filtered in 0.267 seconds, about 225 frames/s. This proves the software and application boundary, not yet the physical optical calibration. *Flat-field capture and verification checklist.* Perform these steps only after the optical alignment checklist above:

- 1) Lock the objective, condenser, lamp voltage, relay, sensor area, exposure, gain, and Bayer phase; record every setting.
- 2) Capture 16–64 full-frame darks with illumination blocked and 16–64 blank-field flats without clipping. Defocus or translate the blank between flat frames so specimen detail does not enter the calibration.
- 3) Build and inspect one `.tca-flat` map. Reject it if the geometry or Bayer phase differs from acquisition, if any plane has a weak reference, or if many pixels require the maximum correction gain.
- 4) Apply the map before demosaic, then estimate global white balance. Preserve the untouched Bayer stream, calibration, corrected derivative, command line, settings, and SHA-256 hashes together.
- 5) Validate a fresh blank quantitatively: compare corner, edge, and center means, inspect seams and clipped pixels, and confirm that correction improves rather than merely hides the asymmetry.
- 6) Check a real specimen as a still and through V4L2. Recalibrate after any change to objective, condenser, adapter, lamp voltage, exposure/gain path, binning, crop, or sensor position.

The complete assembly and white-balance tooling are published at exact commit `0fded6e` as annotated prerelease `v0.2.0-alpha.6`. Its README includes direct FFmpeg recipes for native and scaled stills, live preview, H.264 recording, the neutral-reference estimator, and the existing Linux V4L2 bridge. Alpha 6 also scales all three FFmpeg white-balance gains together when an ideal multiplier exceeds the filter’s 2.0 limit, preserving channel ratios while emitting a valid filter.

The next work is therefore color calibration and application validation rather than basic protocol discovery. A connected-camera Pi reboot produced a new Linux boot ID and USB instance, after which public main rebuilt, passed its tests, and cold-captured three exact-size preview frames with gain and exposure controls. A known color target must still establish the final Bayer phase and color processing, and the corrected reader should be retested through V4L2 on Linux. A controlled `SIGKILL` already demonstrated immediate process-level reopen and three-frame recovery without a device reset. Firmware searches remain useful background work but are

neither a prerequisite nor a reason to delay the working Linux path.

B. Microscopy Workflow Above the Reader

Micro-Manager has two superficially relevant adapters, but only one is a promising path for this camera. Its AmScope adapter is explicitly for Windows-hosted MU-series USB 3 cameras, fixes the offered model to MU1403, and links TouPTek's 2016 SDK[16]; it does not make the C003 bridge a TouPCam device. In contrast, Micro-Manager's Linux Video4Linux2 adapter accepts a configurable `/dev/video*` path and resolution and handles YUYV input[17]. That matches the project's already-proven application boundary. The shortest next integration experiment is therefore the existing reader and temporary V4L2 loopback feeding Micro-Manager, without duplicating the USB protocol.

Pycro-Manager is useful one layer higher: it drives Micro-Manager Core and its configured device adapters rather than supplying an independent camera transport[18]. Once camera and stage devices work in MMCore, its Python acquisition hooks can coordinate Z stacks, XY tiles, processing, and adaptive capture. A sensible first autofocus experiment is a bounded coarse-then-fine Z search on the 1280-by-960 preview, with exposure, gain, illumination, Bayer conversion, and ROI held fixed and variance of the Laplacian used as an initial sharpness score. The final image can then use the full-resolution mode. Automation must wait for a textured specimen and an identified motorized Z stage; the present static dust field is useful for transport and rough focus evidence but not a complete autofocus validation. Also, "a few frames per second" at full resolution is an expectation rather than a measurement: a 10,068,992-byte frame has an overhead-free USB 2.0 signalling ceiling of about 5.96 frames/s, and practical throughput will be lower.

Fiji Grid/Collection Stitching and NIST MIST are credible downstream choices after reproducible overlapping XY acquisition exists. Fiji supports metadata- or layout-guided tile collections and disk-backed large-data operation[19]; MIST is specialized for 2D microscopy mosaics with stage-error modeling and treats time points as independent 2D datasets[20]. The first comparison should use a small 3-by-3 grid with preserved raw tiles, calibrated pixel size, stage coordinates, overlap, Z, exposure, gain, focus score, and timestamps. The complete source comparison and staged recommendation are retained in `evidence/microscopy-application-stack.md`.

C. Release Acceptance Audit

A line-by-line readiness audit separates the useful Linux alpha from final project acceptance. Preview streaming, the portable reader and CLI, the Linux V4L2 application path, safety boundaries, deterministic tests, bounded resource use, reproducible builds, and legal hygiene pass their current criteria. Full sensor transport also passes: the reader captures a coherent 3664 by 2748 Bayer8 raster using validated record-boundary look-ahead assembly.

The microscope session is now specified rather than improvised. The public procedure preserves untouched device and Bayer data while checking all four Bayer phases, orientation, bounded exposure and gain sweeps, a manual focus curve, one full-resolution still, and ordinary-application V4L2 preview. Its analyzer refuses ambiguous frame sizes and output overwrite. Spatially coherent preview/full-resolution stills and monotonic gain response now pass; neutral-background white balance now passes; known-target Bayer/color calibration, a focus curve, and post-fix V4L2 remain.

The physical cable reconnect and microscope-mounted capture have now succeeded. Remaining acceptance decisions concern final Bayer phase and color processing against a known target, focus behavior, measured frame timing, and ordinary-application consumption after the corrected record assembly. The portable Apple Silicon reader has now captured the device both through the Pi relay and directly over the native Mac cable in preview and full-resolution modes. An AVFoundation system-camera surface remains a recorded non-blocking deferral under the Linux-first objective. The complete checklist and evidence mapping are retained in `evidence/release-acceptance-audit.md`.

X. CONCLUSION

The camera is now useful on Linux at an early-prototype level. A portable userspace program cold-initializes the physical 0547:C003 device, applies exposure and gain, receives 1280 by 960 preview or 3664 by 2748 full-resolution Bayer8 frames, and feeds ordinary Linux video applications through a temporary V4L2 device. The camera-specific implementation requires libusb, not a custom kernel driver.

The renewed driver search supports that design. No examined maintained driver matches both the C003 USB bridge and the recovered protocol. The in-tree TouPTek driver is a useful GSPCA and sensor-register reference but binds a different product and sends a different command family. The official TouPCam stable and legacy SDK tables likewise omit C003, and their Python module only wraps the proprietary native library. Current TUCam SDK configuration does not claim C003; the current Micro-Manager TUCam adapter depends on that proprietary SDK; historical `uvscopetek` records the exact model but implements another product; and ArduCAM's MT9J001 files describe the likely sensor rather than the Tucsen bridge. These sources can accelerate carefully reviewed full-resolution and integration work, but none should be bound or replayed wholesale.

The higher-level microscopy suggestion is nevertheless directly actionable. Micro-Manager's AmScope/TouPCam adapter is for another camera family, but its Linux Video4Linux adapter can potentially consume the bridge already supplied by this project. That would make Pycro-Manager, stage automation, preview-mode autofocus, and later Fiji or MIST tile stitching available without changing the working C003 transport.

Process-level repeatability, continuous endurance, host-reboot recovery, physical cable reconnect, and native Apple

Silicon optical capture are now verified. Correct complete-record reads yield coherent preview and full-resolution images; a 100-ms/gain-256 run yielded 16 distinct frames, multi-resolution PNGs, and an H.264 motion proof. Three consecutive fully assembled 3664-by-2748 frames have distinct hashes, and neutral-background white balance removes the measured lamp/sensor cast consistently in both modes. A changed 10x optical configuration was recovered from nearly total clipping to unclipped preview and full-resolution stills using bounded exposure and gain controls. Gain is optically effective and the reader now removes one pre-control buffered frame before publishing frame zero. Linux V4L2 remains the ordinary-application path, while native macOS AVFoundation presentation is a non-blocking later integration. The remaining work is calibration and packaging around a functioning userspace protocol path, not discovery of an unknown kernel driver.

ACKNOWLEDGMENTS

The physical camera, original purchase context, and access to Linux and macOS test systems were provided by Bjørn Remseth.

ABOUT THIS DOCUMENT

This document represents a technical exploration created through a collaborative process between human and AI. The production process followed these steps:

- 1) **Discovery:** A legacy AmScope microscope camera had remained unusable because of its Windows-only software and apparently non-standard USB interface.
- 2) **Initial Exploration:** The owner supplied the approximate purchase date, expected sensor resolution, available hardware, and desired Linux or macOS outcome. No device-specific technical claims were made without USB identification evidence.
- 3) **Synthesis:** The available information was organized into an evidence-gated recovery plan that prefers existing support and user-space prototyping over premature kernel development.
- 4) **Implementation:** Standard USB descriptors were read on macOS and Linux, official legacy installers were preserved and statically extracted, and an isolated Windows guest supplied a working reference implementation. A Raspberry Pi-hosted VirtualHere/usbmon3 run captured the complete reference session. The resulting trace was paired and annotated, then reduced to a narrow libusb startup and streaming implementation. Portable C17 tests cover protocol plans, checked execution, frame assembly, Bayer conversion, replay, controls, and failure paths. Physical Raspberry Pi validation now covers cold initialization, 1280 by 960 continuous capture, exposure and gain, and live application consumption through a removable V4L2 loopback device. Current Linux, Micro-Manager, Pycro-Manager, Tucsen-SDK, historical ScopeTek, MT9J001, Fiji, and MIST sources were then compared against the recovered USB, sensor, acquisition,

and downstream-processing behavior to bound what can safely be reused.

- 5) **Visualization:** Two owner-supplied photographs document the camera enclosure, connector, protection over the sensor area, and absence of a visible model label. Protocol and data-flow diagrams may be added after trace analysis.
- 6) **Verification:** References were checked for authenticity, accessibility, and relevance. The process is documented in the following section.

The document guides reproducible investigation while preserving the evidence needed for implementation and verification.

NOTE ON REFERENCES AND VERIFICATION

This document contains AI-generated content. All external references have been checked for accessibility and matched to the claims they support.

Verification Process:

- Test every URL for accessibility
- Verify author names against real publications
- Confirm DOIs where available
- Validate publication venues
- Check that source content supports the associated claim

Verification Status in References: Each reference includes a `note` field indicating its verification status:

- “Verified: URL accessible” – URL was tested and works
- “Verified: DOI accessible” – DOI was confirmed
- “Standard reference” – Well-known textbook or established work
- “Requires verification” – Needs manual review

Current Status: The cited Tucsen sources are official and were accessible on 2026-09-26. The legacy-camera PDF directly maps the observed VID/PID to the TCA-10.0N and IS1000 names. The current support page establishes only that Tucsen offers a Linux SDK; it does not establish support for this discontinued device. The UCL thesis was checked on 2026-09-29. The Linux, Micro-Manager, Squid/TUCam, uvscopetek, and ArduCAM primary source trees were checked at the cited commits on 2026-10-03. Negative compatibility claims are bounded to those sources and configurations; they are not claims that no private or unpublished driver ever existed. Readers should independently recheck sources used for critical decisions.

REFERENCES

- [1] Tucsen Photonics Co., Ltd., “The discontinued CMOS camera download links,” Official product-support document, verified: official PDF accessible; maps VID 0547 and PID C003 to TCA-10.0N and IS1000 and links their legacy Windows packages. [Online]. Available: <https://www.tucsen.com/uploads/The-Discontinued-CMOS-Camera-Download-Links.pdf>
- [2] Microsoft, “Frequently asked questions about support for windows on arm,” Microsoft Learn, verified: official documentation accessible; states that kernel-mode drivers must be native ARM64 and are not provided by x86/x64 application emulation. [Online]. Available: <https://learn.microsoft.com/windows/arm/faq>
- [3] UTM Project, “USB sharing,” UTM Documentation, verified: official documentation accessible; describes QEMU-backend USB sharing and the macOS hardware-reset limitation. [Online]. Available: <https://docs.getutm.app/guest-support/sharing/usb/>

- [4] AmScope, “Camera support – MU series,” Official camera-support page, verified: lists AmLite for ARM64 Linux dated 2026-04-13 and AmScope software for Windows; downloaded packages were statically inspected and do not claim VID 0547 PID C003. [Online]. Available: <https://amscope.com/pages/camera-support-mu-series>
- [5] Tucsen Photonics Co., Ltd., “Tucsen camera software download,” Official product-support page, verified: official page accessible; lists a Linux SDK kit and the discontinued-camera download document. [Online]. Available: <https://www.tucsen.com/download-software/>
- [6] N. J. Lakhkar, “Phosphate glass microspheres as cell microcarrier substrates for bone tissue engineering applications,” Ph.D. dissertation, University College London, 2014, verified: official UCL thesis PDF accessible; pages 76 and 222–223 identify a Tucsen TCA-10.0-N, Micro-Manager 1.4 live view, and an 80-hour time-lapse workflow. [Online]. Available: https://discovery.ucl.ac.uk/id/eprint/1436722/2/Nilay_Jayant_Lakhkar_PhD_Thesis.pdf_redacted.pdf
- [7] Linux kernel contributors, “ToupTek UCMOS / AmScope MU series camera driver,” Linux kernel source, commit 25d576ed11108470d0999054265108400db1b881, verified: primary source; binds 0547:6801 rather than 0547:C003 and documents a different vendor-request protocol, while exposing related Aptina sensor registers through GSPCA/V4L2. [Online]. Available: <https://github.com/torvalds/linux/blob/25d576ed11108470d0999054265108400db1b881/drivers/media/usb/gspca/touptek.c>
- [8] ToupTek Photonics Co., Ltd., “ToupCamSDK for usb cameras,” Official SDK download centre, verified: official page provides Linux, macOS, Windows, and Python components plus current and legacy archives; static inspection of the 2026-05-19 and 2025-01-20 packages found explicit support for related 0547 products but no 0547:C003 entry. [Online]. Available: <https://www.touptekphotonics.com/download/?category=SDK>
- [9] Cephla Lab, “TUCam Linux USB supported-device configuration,” Squid source repository, commit e1ad4cc6378e69d3315438d6d8839a1e43d573b1, verified: primary source configuration from a current Linux TUCam SDK deployment; lists 0547:A008 and 0547:E004 but not 0547:C003. [Online]. Available: <https://github.com/Cephla-Lab/Squid/blob/e1ad4cc6378e69d3315438d6d8839a1e43d573b1/software/drivers%20and%20libraries/tucsen/sdk/tuusb.conf>
- [10] Micro-Manager contributors, “TUCam device adapter,” Micro-Manager source repository, commit e28c50d10c522c4f2329d3a6a9ac74cb7b7c190e, verified: primary source; Windows x64 adapter linked against the proprietary 2024 TUCam SDK rather than an independent Linux USB implementation. [Online]. Available: <https://github.com/micro-manager/mmCoreAndDevices/tree/e28c50d10c522c4f2329d3a6a9ac74cb7b7c190e/DeviceAdapters/TUCam>
- [11] J. McMaster, “uvscopetek: Open-source microscope camera research,” Source repository, commit db9681b64353c899fff74be28105e6ea19373b53, verified: primary historical source; inventory names IS1000 0547:C003, but implemented transport targets 0547:4D88. [Online]. Available: <https://github.com/JohnDMcMaster/uvscopetek/tree/db9681b64353c899fff74be28105e6ea19373b53>
- [12] ArduCAM, “MT9J001 3664 by 2748 Bayer8 sensor configuration,” ArduCAM USB Camera Shield source repository, commit 23ac88bcbadf2f2652b78a5991980dc11ba6fbf8, verified: primary source; exact full-mode geometry and matching sensor-register vocabulary, but for an unrelated ArduCAM USB bridge. [Online]. Available: https://github.com/ArduCAM/ArduCAM_USB_Camera_Shield/blob/23ac88bcbadf2f2652b78a5991980dc11ba6fbf8/Config/USB2.0_UC-391_Rev.D/DVP/MT9J001/MT9J001_MONO_8b_3664x2748_4fps.cfg
- [13] Nikon MicroscopyU contributors, “Microscope alignment for köhler illumination,” Nikon MicroscopyU tutorial, verified: describes centering and focusing the lamp filament, focusing and centering the condenser with the field diaphragm, and filling the condenser aperture for even specimen illumination. [Online]. Available: <https://www.microscopyu.com/tutorials/kohler>
- [14] —, “Conjugate planes in optical microscopy,” Nikon MicroscopyU microscopy-basics article, verified: explains Köhler illumination, objective-rear-aperture inspection for troubleshooting, and the usual 70–80 percent condenser-aperture setting. [Online]. Available: <https://www.microscopyu.com/microscopy-basics/conjugate-planes-in-optical-microscopy>
- [15] Micro-Manager contributors, “Flat-field correction,” Official Micro-Manager documentation, verified: documents averaged dark acquisition, multi-position flat acquisition, and correction by dividing acquired-minus-background by flat-minus-background. [Online]. Available: https://micro-manager.org/Flat-Field_Correction
- [16] —, “AmScope MU-Series device adapter,” Micro-Manager source repository, commit e28c50d10c522c4f2329d3a6a9ac74cb7b7c190e, verified: primary source; Windows-only adapter for AmScope MU-series USB 3 cameras, fixed to MU1403 in source and linked to the 2016 ToupCam SDK, not a transport for 0547:C003. [Online]. Available: <https://github.com/micro-manager/mmCoreAndDevices/tree/e28c50d10c522c4f2329d3a6a9ac74cb7b7c190e/DeviceAdapters/AmScope>
- [17] —, “Video4Linux2 device adapter,” Micro-Manager source repository, commit e28c50d10c522c4f2329d3a6a9ac74cb7b7c190e, verified: primary source; Linux camera adapter with configurable device path and resolution and YUYV handling, matching the project’s proven V4L2 loopback boundary. [Online]. Available: <https://github.com/micro-manager/mmCoreAndDevices/blob/e28c50d10c522c4f2329d3a6a9ac74cb7b7c190e/DeviceAdapters/Video4Linux/video4linux2.cpp>
- [18] Pycro-Manager contributors, “Pycro-manager backends and micro-manager core architecture,” Official documentation, verified: documents Pycro-Manager as a Python acquisition layer using Micro-Manager Core through Java or Python backends rather than an independent camera transport. [Online]. Available: <https://pycro-manager.readthedocs.io/en/latest/backends.html>
- [19] ImageJ contributors, “Grid/collection stitching plugin,” Official ImageJ/Fiji documentation, verified: documents tile-grid and collection stitching, metadata-defined positions, overlap refinement, blending, and memory-management options. [Online]. Available: <https://imagej.net/plugins/grid-collection-stitching>
- [20] National Institute of Standards and Technology, “MIST: Microscopy image stitching tool,” Source repository, commit 26481de8cccf7f7c58ba86549556e8544368d1e4, verified: primary source; NIST 2D microscopy stitching tool with stage modeling and time-series support as independent 2D datasets, not volumetric 3D stitching. [Online]. Available: <https://github.com/usnistgov/MIST/tree/26481de8cccf7f7c58ba86549556e8544368d1e4>